# 🚀 Terraform Fundamentals

## The Complete Visual Study Guide for Infrastructure as Code

### 🎯 What is this Study Guide?

These handwritten-style notes distill everything you need to master **HashiCorp Terraform** from Day 0 to Production. Master IaC mindset, syntax, providers, state locking, and best practices!

### 📌 What We Are Covering (The 20-Step Journey):

- **IaC Mindset** & Declarative vs Imperative provisioning
- The Golden Workflow: **Write → Plan → Apply**
- HCL Anatomy, Providers, Resources vs Data Sources
- **Terraform State** internals, Remote backends & S3/DynamoDB locks
- Variables, Locals, Outputs, Modules & Meta-arguments

### 🎨 Terraform Architecture at a Glance



## Why Infrastructure as Code?

Before IaC, engineers built infrastructure manually via the Cloud Console (ClickOps). This led to severe configuration drift, zero reproducibility, and human error.

### 🔑 Key Benefits of IaC:

- **Version Control:** Track infra changes in Git with PR reviews.
- **Consistency:** Exact duplicate environments (dev, stage, prod).
- **Speed & Safety:** Automated testing, fast rollbacks & CI/CD.

### Declarative vs Imperative:

Terraform is Declarative: You define **WHAT** end state you want, Terraform figures out **HOW** to achieve it.

⚔️ Imperative (Scripting) vs Declarative (Terraform)

**Imperative (e.g. Bash/CLI)**

"Focus on HOW"

1. Check if VM exists
2. If no, create VM
3. Attach EBS volume
4. Configure security group

⚠○ High maintenance

**Declarative (Terraform)**

"Focus on WHAT"

```
resource "aws_instance" {
  count = 3
  type = "t3.micro"
}
```

✅ Self-healing State

💡 **Core Concept:** Declarative means if you delete a resource from HCL, Terraform knows to delete it in the cloud automatically.

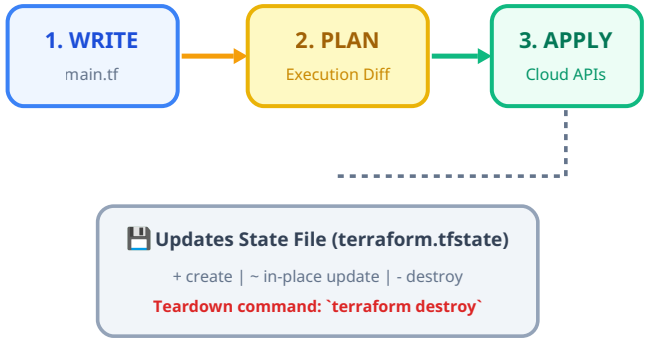
## The Golden Trinity: Write ➔ Plan ➔ Apply

Every Terraform operation revolves around this 3-stage feedback cycle:

1. **WRITE (Code HCL):** Author your infrastructure definitions in `.tf` files using declarative syntax.
2. **PLAN (Dry Run):** Run `terraform plan`. It compares HCL with state and cloud APIs, outputting an execution diff (+, ~, -).
3. **APPLY (Provision):** Run `terraform apply`. Performs actual API calls and writes the new state file.

CRITICAL: Never apply changes directly in production without carefully inspecting the **Plan output**!

🔄 The Terraform Feedback Cycle Diagram




💡 **Pro-tip:** In CI/CD pipelines, save plan to a file: `terraform plan -out=tfplan` and apply the exact artifact: `terraform apply tfplan`.

## Two-Tier Architecture: Core & Plugins

Terraform is split into two independent decoupled systems communicating over **gRPC (RPC)**:

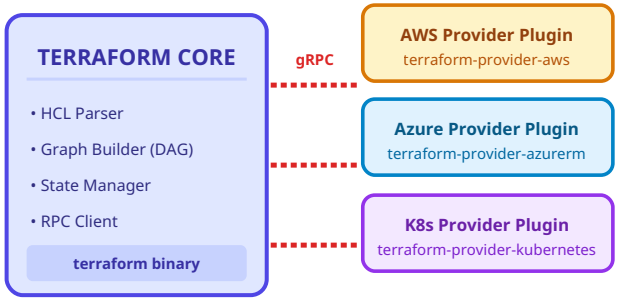
### 1. Terraform Core (Engine):

- Parses `.tf` configuration and modules.
- Constructs the Directed Acyclic Graph (DAG).
- Manages State reconciliation & calculations.
- Invokes providers via standardized RPC calls.

### 2. Terraform Plugins (Providers):

- Executable binaries downloaded during `terraform init`.
- Translates RPC messages into **Cloud API requests**.

🔌 Terraform Core & gRPC Provider Plugin Model




💡 **Why decouple?** Providers can be updated and released independently from the core Terraform engine!

## Anatomy of a Resource Block

HashiCorp Configuration Language ( **HCL** ) is declarative, human-readable, and structured around blocks, arguments, and expressions.

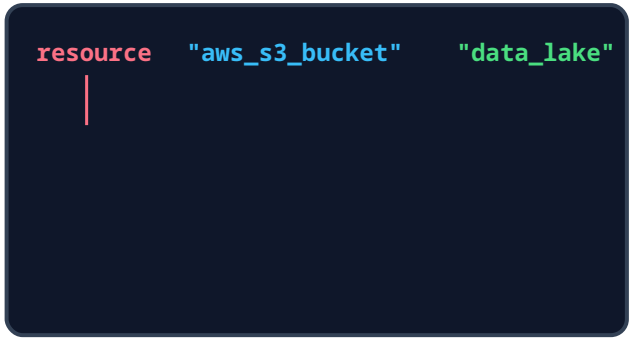
```
resource "aws_instance" "web_server" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.micro"

  tags = {
    Name        = "Production-Web"
    Environment = "Production"
  }
}
```

### Block Components:

- **Block Type:** `resource` (Defines what kind of block).
- **Resource Type:** `aws_instance` (Provided by AWS plugin).
- **Local Name:** `web_server` (Internal reference identifier).

🔍 Visual Dissection of HCL Resource Block




aws_s3_bucket.data_lake.id Format:
<RESOURCE_TYPE>.<LOCAL_NAME>.<ATTRIBUTE>

## Provider Configuration & Aliases

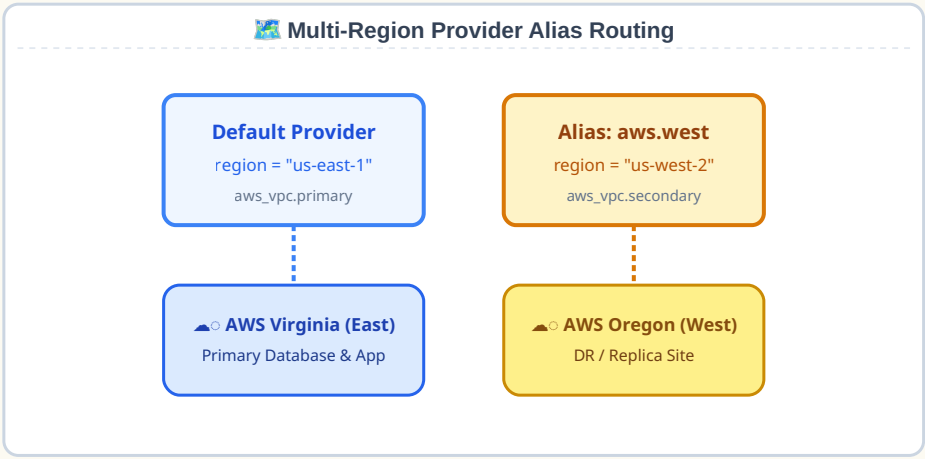
Providers manage the lifecycle of resources. You declare required versions in the `terraform` block.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

# Secondary Provider Alias for Multi-Region
provider "aws" {
  alias  = "west"
  region = "us-west-2"
}
```

Pass alias to resources: `provider = aws.west`




💡 **Best Practice:** Always pin provider versions using `~>` (pessimistic operator) to prevent unexpected breaking updates!

## Creating vs Fetching Existing State

**🏗️ Resource Block (Write / Manage):**

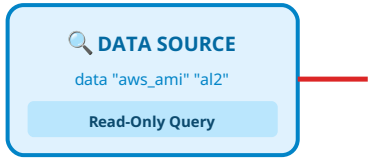
Creates, modifies, or destroys infrastructure. Managed completely inside Terraform state.

**🔍 Data Source Block (Read-Only):**

Queries existing infrastructure created outside Terraform (or by another team) without managing its lifecycle.

```
# Query latest Amazon Linux 2 AMI
data "aws_ami" "latest_amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}
```

🔄 Data Source to Resource Pipeline



## The Heart of Terraform: State File

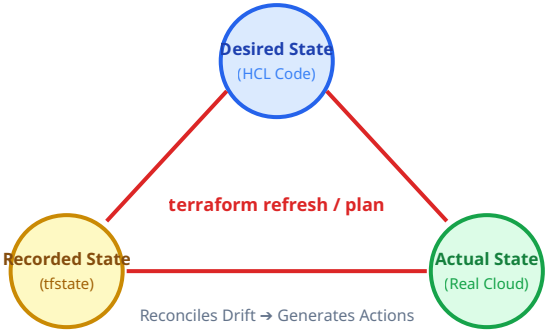
Terraform creates a JSON file called **terraform.tfstate**. It maps declarative HCL configurations to real-world cloud IDs.

### Why is State Essential?

- **Resource Mapping:** Tracks that `aws_instance.web` is `i-0a8b9c1d2e`.
- **Performance:** Caches cloud metadata for fast graph rendering.
- **Dependency Tracking:** Stores creation order for teardowns.

⚠️ WARNING: The state file contains **unencrypted sensitive data** (database passwords, private keys)!

⚖️ The 3-Way State Reconciliation Triangle




💡 **Rule #1:** NEVER manually edit `terraform.tfstate` in a text editor! Use CLI state commands instead.

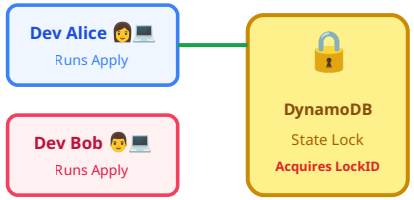
## S3 Backend + DynamoDB State Locking

Storing state locally on laptops creates collisions and race conditions. In production, use a **Remote Backend** with distributed state locking.

```
terraform {
  backend "s3" {
    bucket         = "company-tf-state-prod"
    key            = "vpc/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "tf-state-lock-table"
  }
}
```

**DynamoDB Lock:** Creates a unique `LockID` when anyone runs `plan/apply`, preventing simultaneous updates.

🔒 Production Remote State Architecture



## Parameterizing Infrastructure

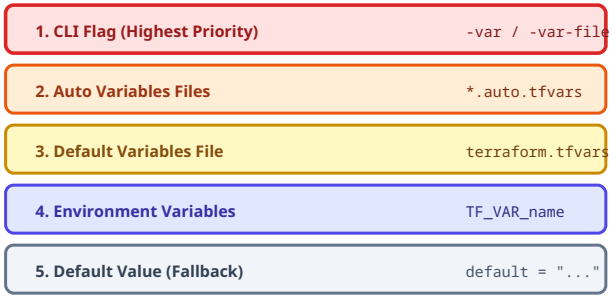
Variables make configurations dynamic and reusable across environments.

```
variable "db_port" {
  type        = number
  default     = 5432
  description = "Port for PostgreSQL cluster"
}

variable "db_password" {
  type        = string
  sensitive   = true  # Masked in logs

  validation {
    condition     = length(var.db_password) >= 8
    error_message = "Password must be >= 8 chars."
  }
}
```

### 🪜 Variable Definition Precedence Ladder




Higher level values override everything beneath them!

💡 **Security Note:** Setting `sensitive = true` hides values in CLI output, but it is STILL stored in plaintext inside state!

## Locals vs Variables vs Outputs

### 1. Local Values (`locals`):

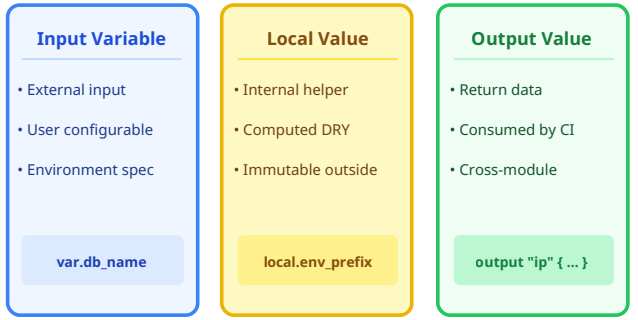
Internal constants or transformed expressions to avoid repetition (DRY). Cannot be overridden from outside.

```
locals {
  env_prefix = "${var.project}-${var.environment}"
  common_tags = {
    Owner     = "DevOps-Team"
    ManagedBy = "Terraform"
  }
}
```

### 2. Output Values (`output`):

Expose return values to CLI or pass attributes to other root/child modules.

### ⚖️ Quick Reference: Inputs vs Locals vs Output




💡 **CLI Tip:** Query outputs anytime without running plan using: `terraform output -json`.

## The Daily Driver CLI Commands

| Command | Purpose & Action |
|---|---|
| `terraform init` | Downloads plugins, initializes backend. |
| `terraform fmt -recursive` | Formats HCL to canonical standards. |
| `terraform validate` | Checks syntax and internal consistency. |
| `terraform plan` | Dry-run diff against cloud state. |
| `terraform apply -auto-approve` | Executes actions directly (Use caution!). |
| `terraform console` | Interactive REPL to test expressions. |

💻 Terminal Workflow Simulation

```
$ terraform plan
Terraform used the selected providers...
+ create aws_instance.web
~ update aws_security_group.allow_http
- destroy aws_s3_bucket.legacy
----------------------------------------------------------------
Plan: 1 to add, 1 to change, 1 to destroy.

$ terraform apply "tfplan"
Apply complete! Resources: 1 added.
```

💡 **Clean Code:** Always add `terraform fmt -check` to pre-commit git hooks and CI/CD pipelines!

## Implicit vs Explicit Dependencies

Terraform builds a **Directed Acyclic Graph (DAG)** to provision independent resources in parallel for maximum speed.

### 1. Implicit Dependency (Automatic):

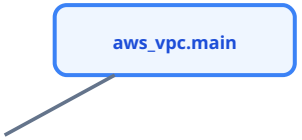
Created whenever you reference an attribute from another resource.

```
subnet_id = aws_subnet.public.id
```

### 2. Explicit Dependency (`depends_on`):

Used when resources depend on each other without direct attribute references (e.g. IAM role propagation, S3 bucket logging).

🌳 DAG Parallel Execution Tree



## Iterators & Resource Lifecycle

**1. `count` vs `for_each`:**

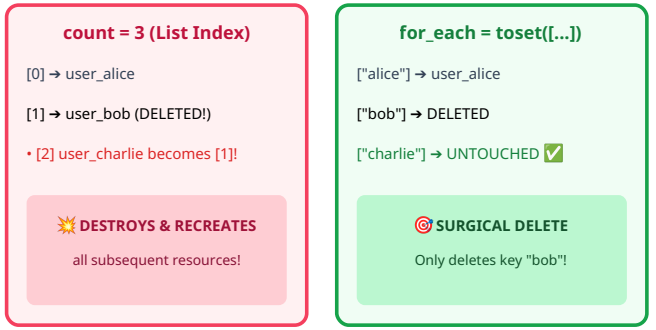
**count:** Uses integer indexes (0, 1, 2). *Risk:* Deleting item #1 forces recreation of all following items!

**for_each:** Uses Map/Set keys. **Safe!** Adding or removing elements does not shift sibling indexes.

**2. The `lifecycle` Block:**

- `create_before_destroy = true` (Zero downtime replacement)
- `prevent_destroy = true` (Guard production DBs!)
- `ignore_changes = [tags["AutoUpdated"]]`

### ⚠️ The "Count Index Shift" Trap vs "For_Each"




💡 **Golden Standard:** Use `for_each` by default for infrastructure resources; reserve `count` strictly for simple boolean toggles (`count = var.enabled ? 1 : 0`).

## Building Reusable Blueprints

A module is a container for multiple resources used together. Think of modules as **reusable Lego blocks** for your cloud architecture.

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.1.0"

  name = "prod-vpc"
  cidr = "10.0.0.0/16"

  azs             = ["us-east-1a", "us-east-1b"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24"]

  enable_nat_gateway = true
}
```

### 📁 Standard Module Directory Layout

```
modules/aws-secure-vpc/
├── main.tf (Core resource definitions)
├── variables.tf (Configurable inputs)
├── outputs.tf (Exposed subnet IDs, VPC ID)
├── README.md (Documentation & examples)
└── versions.tf (Minimum terraform & provider specs)
```

**Root Module** → Calls → **Child Modules** (VPC, EKS, RDS)
Encapsulates complexity so developers only pass essential params!

💡 **Reusability:** Avoid massive 2,000-line single `main.tf` files. Decompose architecture into domain-specific modules.

## Dynamic Blocks for Nested Arguments

Construct repetitive nested configuration blocks (like firewall ingress rules or ECS port mappings) programmatically:

```
resource "aws_security_group" "web" {
  name = "web-sg"

  dynamic "ingress" {
    for_each = [80, 443, 8080]
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

### 🧰 Essential Built-in Functions Cheat Sheet

| Function | Example / Return |
|---|---|
| `lookup(map, key, def)` | Safe dictionary lookups |
| `merge(map1, map2)` | Combines tags / maps |
| `flatten(list_of_lists)` | Flattens multidimensional lists |
| `file("${path.module}/init.sh")` | Reads local file into string |
| `templatefile(path, vars)` | Renders dynamic templates |
| `jsonencode(obj)` | Converts HCL map to JSON string |

💡 **Test in REPL:** Run `terraform console` to experiment and test functions like `merge()` in real-time.

## Workspaces vs Directory Separation

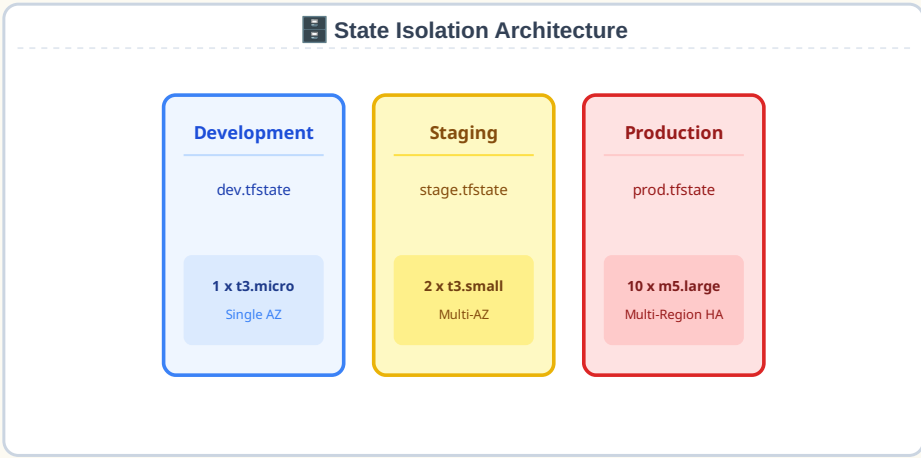
### 1. Terraform Workspaces:

Stores separate state files in the same backend (`terraform.workspace` interpolation).

```
count = terraform.workspace == "prod" ? 5 : 1
```

### 2. Directory-Based Separation (Recommended for Prod):

Separate folders (`envs/dev`, `envs/prod`) with distinct backend configs and IAM credentials. Total fault isolation!




💡 **Industry Verdict:** Workspaces work well for identical ephemeral testing branches; Use Directory separation for Dev vs Prod to prevent accidental blast radius.

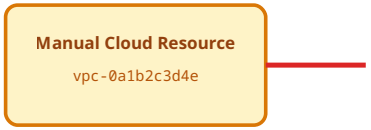
## Performing Safe State Surgery

| State Command | Action Description |
|---|---|
| `state list` | List all tracked resources in state |
| `state show <addr>` | Inspect full metadata of 1 resource |
| `state mv <src> <dst>` | Rename/refactor resource without deleting! |
| `state rm <addr>` | Untrack resource (keeps alive in cloud) |
| `import <addr> <id>` | Bring existing manual cloud asset into state |

### 🛡️ The Golden Safety Rule:

Always run `terraform state pull > backup.tfstate` before executing any manual state mutation commands!

### 🔄 Adopting Legacy Infrastructure (`terraform import`)



## Securing Infrastructure as Code

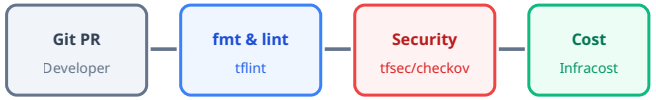
### 🚨 The 3 Deadly Security Sins:

- **Hardcoding AWS Access Keys** in code (Commit to GitHub = Immediate breach!).
- Leaving S3 state bucket public and unencrypted.
- Not enabling DynamoDB state locking in multi-user teams.

### Static Analysis & Security Scanners:

- **tfsec / Trivy:** Scans HCL for open security groups, unencrypted EBS, etc.
- **Checkov:** Policy-as-code enforcement.
- **Infracost:** Cloud cost estimate directly in GitHub PRs!

### 🛡️ Automated DevSecOps CI/CD Pipeline




**🔐 Secrets Management Best Practice**

Use HashiCorp Vault or AWS Secrets Manager dynamically

**Never put credentials in *.tf or *.tfvars files!**

💡 **Git Safety:** Add `*.tfstate`, `*.tfstate.backup`, and `*.tfvars` to your `.gitignore` immediately!

# 📓 Topic 19: The 10 Golden Rules of Production Terraform

Final Master Checklist

Jatin Kishore M

20 / 20

## The 10 Commandments of Terraform

1. Always use a **Remote Backend** with S3 encryption & DynamoDB locking.
2. Pin both **Terraform core** and **Provider versions** explicitly.
3. Prefer `for_each` over `count` to avoid destructive index shifts.
4. Never manually edit `terraform.tfstate`.
5. Always save & review plan files in CI/CD before applying.
6. Protect database and critical state with `prevent_destroy = true`.
7. Decompose monolithic stacks into decoupled micro-modules.
8. Enforce automated `terraform fmt` and security scans in Git hooks.
9. Use `data` sources instead of hardcoding foreign cloud resource IDs.
10. Isolate environments (Dev vs Prod) in separate directories and accounts.

🎓

### CONGRATULATIONS!

You have mastered the foundational building blocks of **HashiCorp Terraform**!

**Verified Terraform Study Notes**

Author Brand: **Jatin Kishore M**

✅ **20 / 20 Pages Complete & Validated**

Ready for LinkedIn Carousel / Slide Deck Publication!

⭐ **Next Steps:** Practice building a 3-tier AWS architecture using modules and remote state locking!